ADIMAT POS

Adimat POS

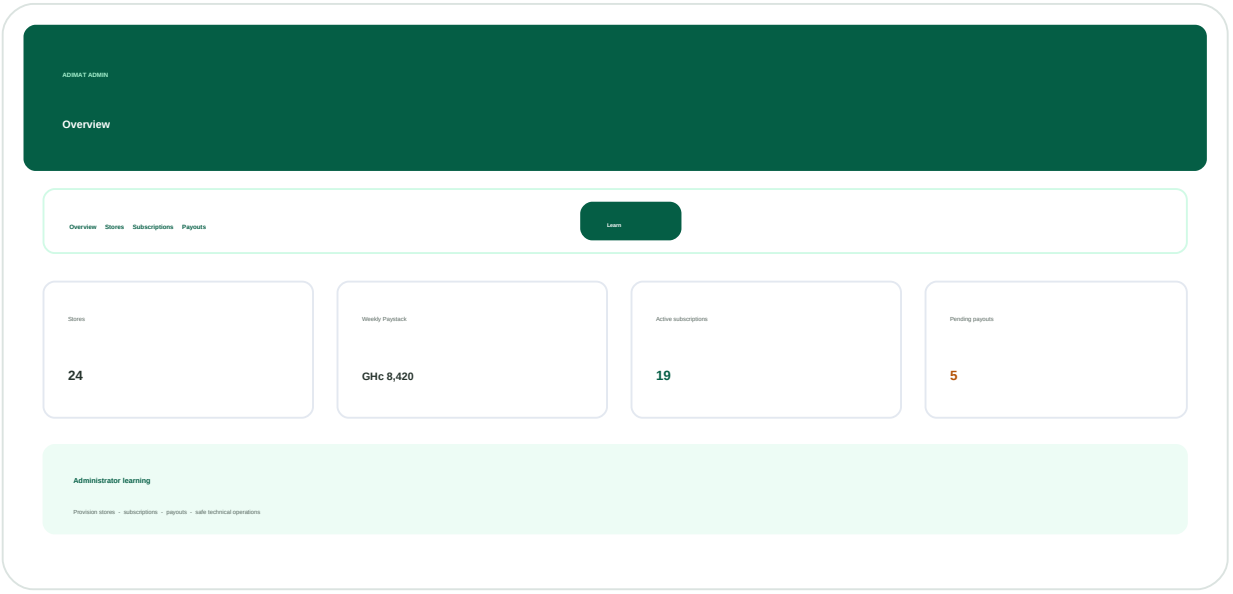
PLATFORM ADMINISTRATOR AND TECHNICAL STAFF GUIDE

A practical guide for using Adimat POS

Edition: 12 October 2026

Learn Adimat

Illustrated step-by-step instructions for the features available to your role.



ADMIN PORTAL

Administrator portal illustration. Use the Learn tab after secure administrator sign-in.

For platform administrators and technical staff responsible for provisioning, deployment, database operations, and support.

Administrator portal overview

Sign in through the administrator portal. The portal requires the configured administrator PIN, Supabase Auth SMS verification, and the trusted platform-administrator role. Use the Learn tab for this operational guide; store owners and cashiers have different guides inside their store app.

Provision and manage stores

1. In Stores, create a store and securely deliver the Store ID and owner PIN to its owner. Avoid sending credentials in a public channel.
2. In Subscriptions, grant or review store access. Set warehouse quotas or storefront options only when included in the store's plan and workflow.
3. In Contact, maintain administrator contact details and SMS notification configuration.
4. Review Reports and Payouts for platform-level summaries and weekly remittance status. Validate store Mobile Money details before marking a payout as sent.
5. Clear trial sales only after confirming the target store and obtaining approval. The operation deletes sales and related ledger records, keep a data backup and ask store staff to run a full sync afterward.

Build and deploy

Use Node.js 18+ and npm. From the project root, run , configure environment variables for the build, run , and deploy to static hosting over HTTPS. allows a local check of the production output.

Client build variables include , and optionally , administrator PIN configuration, and SMS defaults. Values prefixed with are embedded in browser-delivered code. Never put a Supabase service-role key, payment secret key, or private credential in the client build or source control.

Supabase schema and migrations

For a new Supabase project, apply . Existing projects require only migrations relevant to their current schema and enabled features. Common feature migrations include:

- , , , for admin, plans, and renewal flows.
- for campaign tooling.
- , , and for locations, limits, supplier orders, movement history, and reorder levels.
- and for expense automation and online orders.
- , , and for their corresponding features.

Review the SQL and dependencies against the actual deployed schema; do not blindly apply every migration. Back up production data first. After applying a migration, verify the columns/tables and run a test store through the affected workflow and cloud sync.

Sync, backups, and device support

Adimat stores business data locally in IndexedDB. Full Sync pulls cloud changes and then pushes local changes; Pull and Push perform only their named direction. Check last-sync status and pending changes before moving a store to another device.

Automatic snapshots are on-device only. Downloaded JSON backups may include sensitive staff records and PIN hashes. Keep a separate secure backup, limit access, and verify the selected store before restore. Restore replaces local data for the selected store. Avoid clearing browser storage during troubleshooting because unsynced records may be lost.

For a store that has been reset by an administrator, ask its staff to update/reload the app and run a full sync before reviewing Sales or Reports. For a device/peripheral issue, collect the browser and device model, exact steps, time, and error text; do not request PINs or backup exports over an insecure channel.

Production security

The application is a static client with local PIN authentication. The client bundle contains the public Supabase anon key. Current database policies documented in the system reference are permissive and do not independently isolate tenants. UI role checks and store IDs are not server-side authorization.

Before untrusted multi-tenant production use, implement and test restrictive server-side RLS and authorization for every tenant-scoped table and storage resource. Harden cloud restore and administrator actions. Verify that ordinary store users cannot access another tenant's data through direct API requests. Keep HTTPS enabled, restrict administrator accounts, rotate exposed credentials, and never deploy service-role credentials to browsers.

Release verification

After schema or app releases, validate owner and cashier access independently, cash sales, Paystack if enabled, inventory movements, cloud pull/push, backup/restore, and enabled warehouse and e-commerce functions. Review sync errors and pending changes. For deeper architecture, database model, and module detail, read .